Introduction to Interfaces

What Is an Interface?

An interface in Java is a reference type that defines a contract that implementing classes agree to follow. It is used to achieve abstraction and support multiple inheritance of type.

An interface can declare abstract methods, constants, and—depending on the Java version—default, static, and private methods.

Definition for exams: An interface is a reference type in Java that specifies a set of methods and constants that classes can implement to provide a common contract.

Syntax of an Interface

```
interface InterfaceName {
    void display();
}
```

A class implements an interface using the `implements` keyword.

```
class Demo implements InterfaceName {
    public void display() {
        System.out.println("Interface implemented");
    }
}
```

Example of an Interface

```
interface Animal {
    void sound();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}

class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output:

```
Dog barks
```

Explanation

1. `Animal` is an interface.
2. `sound()` is an abstract method declared in the interface.
3. `Dog` implements the `Animal` interface.
4. `Dog` provides the implementation of `sound()`.
5. The method is declared `public` because interface abstract methods are implicitly public.

Features of Interfaces

- An interface is declared using the `interface` keyword.
- Interfaces cannot be instantiated directly.
- A class uses `implements` to implement an interface.
- A class can implement more than one interface.
- Interface fields are implicitly `public static final`.
- Abstract methods declared in an interface are implicitly `public abstract`.
- Interfaces can extend one or more interfaces.
- Interfaces can contain `default` and `static` methods; modern Java also supports private helper methods.
- Interfaces help achieve abstraction and loose coupling.

Advantages of Interfaces

1. Abstraction: Define required behavior without specifying every implementation detail.
2. Multiple inheritance of type: A class can implement multiple interfaces.
3. Code flexibility: Different classes can provide different implementations of the same interface.
4. Loose coupling: Code can depend on an interface rather than a particular implementation.
5. Maintainability: Implementations can change without necessarily changing code that uses the interface.

Example: Multiple Interfaces

Java does not allow a class to extend multiple classes, but it allows a class to implement multiple interfaces.

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

class Demo implements Printable, Showable {
    public void print() {
        System.out.println("Printing");
    }

    public void show() {
        System.out.println("Showing");
    }
}

class Test {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.print();
        d.show();
    }
}
```

Output:

```
Printing
Showing
```

Extending an Interface

An interface can inherit the methods and other members of another interface by using the `extends` keyword.

Definition for exams: Extending an interface is the process by which one interface inherits the contract of another interface using the `extends` keyword.

Unlike a class, an interface can extend more than one interface.

Syntax

```
interface A {
    void methodA();
}

interface B extends A {
    void methodB();
}
```

Here, interface `B` extends interface `A` and inherits its contract.

Example of Extending an Interface

```
interface Animal {
    void eat();
}

interface Dog extends Animal {
    void bark();
}

class Puppy implements Dog {
    public void eat() {
        System.out.println("Puppy eats food");
    }

    public void bark() {
        System.out.println("Puppy barks");
    }
}

class Test {
    public static void main(String[] args) {
        Puppy p = new Puppy();

        p.eat();
        p.bark();
    }
}
```

Output:

```
Puppy eats food
Puppy barks
```

Explanation

1. `Animal` declares the `eat()` method.
2. `Dog` extends `Animal` and declares `bark()`.

3. `Puppy` implements `Dog`.
4. Since `Dog` extends `Animal`, `Puppy` must also implement `eat()`.
5. The `Puppy` object can call both methods.

Extending Multiple Interfaces

An interface can extend multiple interfaces separated by commas.

Example:

```
interface A {
    void displayA();
}

interface B {
    void displayB();
}

interface C extends A, B {
    void displayC();
}

class Demo implements C {
    public void displayA() {
        System.out.println("Interface A");
    }

    public void displayB() {
        System.out.println("Interface B");
    }

    public void displayC() {
        System.out.println("Interface C");
    }
}

class Test {
    public static void main(String[] args) {
        Demo d = new Demo();

        d.displayA();
        d.displayB();
        d.displayC();
    }
}
```

Output:

```
Interface A
Interface B
Interface C
```

Exam point: A class uses `extends` to inherit from a class, an interface uses `extends` to inherit from another interface, and a class uses `implements` to implement an interface.

Implementing an Interface

Implementing an interface means that a class provides the implementation of the abstract methods declared by that interface.

The `implements` keyword is used to connect a class with an interface.

A class implements an interface by using the `implements` keyword and providing implementations for the interface's required abstract methods.

Syntax

```
class ClassName implements InterfaceName {
    public void methodName() {
        // Method implementation
    }
}
```

Example of Implementing an Interface

```
interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle implements Shape {
    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}

class Test {
    public static void main(String[] args) {
        Circle c = new Circle();
        Rectangle r = new Rectangle();

        c.draw();
        r.draw();
    }
}
```

Output:

```
Drawing Circle
Drawing Rectangle
```

Explanation

- `Shape` declares the `draw()` method.
- `Circle` and `Rectangle` implement the same interface.
- Each class provides its own implementation.
- The same interface contract supports different behaviors.

Rules for Implementing an Interface

1. Use the `implements` keyword.
2. A concrete class must implement all inherited abstract methods.
3. Implemented interface methods must be declared `public`.
4. If a class does not implement all inherited abstract methods, it must be declared `abstract`.
5. A class can implement multiple interfaces.
6. A class can extend one superclass and implement multiple interfaces.

Example:

```
class Demo extends Parent implements A, B {
    // Implement required methods
}
```

Here, `Parent` is a superclass, while `A` and `B` are interfaces.

Difference Between Extending and Implementing an Interface

Extending an Interface	Implementing an Interface
Uses the `extends` keyword.	Uses the `implements` keyword.
An interface inherits another interface's contract.	A class provides implementations of an interface's abstract methods.
An interface can extend multiple interfaces.	A class can implement multiple interfaces.
Example: `interface B extends A`	Example: `class C implements A`

3.6 Interface vs. Abstract Class

Interface	Abstract Class
Declared using `interface`.	Declared using `abstract class`.
Cannot be instantiated directly.	Cannot be instantiated directly.
Fields are implicitly `public static final`.	Can have instance variables and other field types.
Has no constructors.	Can have constructors.
A class can implement multiple interfaces.	A class can extend only one class.
Useful for defining a contract.	Useful for a common base with shared state and behavior.

Accessing Interface Variables

What Are Interface Variables?

Variables declared inside an interface are implicitly:

- `public`
- `static`
- `final`

This means interface variables are public constants associated with the interface, not separate instance variables belonging to objects.

Interface variables are constants declared inside an interface that are automatically `public`, `static`, and `final`.

Syntax

```
interface InterfaceName {
    int VALUE = 100;
}
```

The declaration above is equivalent to:

```
interface InterfaceName {
    public static final int VALUE = 100;
}
```

The keywords are optional because Java supplies these modifiers implicitly.

Accessing Interface Variables Using the Interface Name

Example:

```
interface Student {
    int MARKS = 90;
}

class Test {
    public static void main(String[] args) {
        System.out.println(Student.MARKS);
    }
}
```

Output:

```
90
```

Here, `MARKS` is accessed using `Student.MARKS`. No object is required.

Accessing Interface Variables Through an Implementing Class

Example:

```
interface College {
    String NAME = "MJPRU";
}

class Student implements College {
    void display() {
        System.out.println(NAME);
        System.out.println(College.NAME);
        System.out.println(Student.NAME);
    }
}

class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Output:

```
MJPRU
MJPRU
MJPRU
```

All three references access the same interface constant. The clearest form is `College.NAME`.

Rules for Interface Variables

1. Every interface variable is implicitly `public static final`.
2. An interface variable must be initialized when declared.

3. Its value cannot be reassigned.
4. It can be accessed using the interface name.
5. An implementing class can refer to an accessible interface constant.
6. An interface variable is not an instance variable of the implementing class.

For example, this statement is illegal:

```
interface Demo {
    int VALUE = 10;
}

// Demo.VALUE = 20; // Compilation error
```

The variable is final and cannot be reassigned.

Exam tip: If asked "How are interface variables accessed?", explain their `public static final` nature and demonstrate `InterfaceName.VARIABLE_NAME` in a complete program.

Packages in Java

A package in Java is a namespace that groups related classes, interfaces, and other package-level types together.

Packages help organize programs, avoid naming conflicts, and control access.

A package is a collection of related classes and interfaces organized under a common name to provide better organization, reusability, and access control.

Types of Packages

Java packages are commonly classified into two categories.

A. Built-in Packages

These are packages provided by the Java platform.

Examples:

Package	Purpose	Example
`java.lang`	Core language classes	`String`, `Math`, `System`
`java.util`	Utilities and collections	`Scanner`, `ArrayList`
`java.io`	Input/output operations	`File`, `BufferedReader`
`java.net`	Networking	`URL`, socket classes
`java.time`	Date and time	`LocalDate`, `LocalTime`

The `java.lang` package is automatically imported into ordinary Java source files.

B. User-Defined Packages

These are packages created by programmers to organize their own classes and interfaces.

For example:

```
package mypack;
```

```
public class Demo {
    public void display() {
        System.out.println("User-defined package");
    }
}
```

The statement `package mypack;` declares that `Demo` belongs to the `mypack` package.

Advantages of Packages

1. Organize related classes and interfaces.
2. Avoid naming conflicts between types.
3. Support access control.
4. Improve code reusability.
5. Make large applications easier to maintain.
6. Help separate program components into logical groups.

Creating a User-Defined Package

Consider the following file named `Demo.java`:

```
package mypack;

public class Demo {
    public void display() {
        System.out.println("Welcome to my package");
    }
}
```

Save the file in the appropriate package directory.

Compile it from the parent directory:

```
javac -d . Demo.java
```

This creates the package directory structure and places the compiled class in the appropriate location.

Importing and Using a Package

Create a second file named `Test.java` outside the `mypack` package:

```
import mypack.Demo;

class Test {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.display();
    }
}
```

Compile and run from the same parent directory:

```
javac -d . Demo.java
javac -cp . Test.java
java -cp . Test
```

Output:

```
Welcome to my package
```

Explanation:

1. `package mypack;` declares the package.
2. `public class Demo` makes the class accessible from other packages, subject to normal access rules.
3. `import mypack.Demo;` allows the simple class name `Demo` to be used.
4. The object is created and the `display()` method is called.

The import Keyword

The `import` keyword allows a type or types from another package to be referenced by their simple names.

Import a Single Class

```
import java.util.Scanner;
```

Import All Accessible Types in a Package

```
import java.util.*;
```

The wildcard import does not import subpackages, and it does not load every class in the package at runtime.

Use a Fully Qualified Name

You can use a class without an import statement by writing its fully qualified name:

```
java.util.Scanner sc = new java.util.Scanner(System.in);
```

Package Declaration Rules

1. A package declaration, if present, must appear before import declarations and type declarations.
2. A source file can have at most one package declaration.
3. An `import` statement follows the package declaration, if present.
4. The directory structure normally corresponds to the package name.
5. Public classes intended for use outside their package must be accessible and properly compiled.
6. Classes in different packages may have the same simple name; their package names distinguish them.

Difference Between Built-in and User-Defined Packages

Built-in Package	User-Defined Package
Provided by the Java platform.	Created by the programmer.
Contains standard library types.	Contains application-specific types.
Example: `java.util`.	Example: `mypack`.
Used by importing or qualifying standard classes.	Declared using the `package` statement and used through imports or qualified names.

Quick Revision Sheet

Topic	Key point
Interface	Defines a contract for implementing classes.
`interface`	Keyword used to declare an interface.
`extends`	Used by an interface to inherit from another interface.
`implements`	Used by a class to implement an interface.
Multiple interfaces	A class can implement multiple interfaces.
Interface variables	Implicitly `public static final`.
Built-in package	Example: `java.util`.
User-defined package	Example: `mypack`.
`package`	Declares a package for a type.
`import`	Allows types to be referenced by their simple names.

Introduction to Java API

What is Java API?

Java API stands for Java Application Programming Interface. It is a collection of predefined classes, interfaces, and other library components that programmers can use to develop Java applications.

The Java API provides ready-made functionality for common programming tasks, such as input/output, string processing, mathematics, collections, networking, and database connectivity.

Features of Java API

1. Provides predefined classes and interfaces.
2. Reduces programming effort.
3. Supports code reusability.
4. Provides functions for common operations.
5. Organizes related classes into packages.
6. Helps developers write maintainable programs.

Important Java API Packages

Package	Purpose	Examples
`java.lang`	Fundamental language functionality	`String`, `Math`, `System`, `Object`
`java.util`	Utilities and collections	`Scanner`, `ArrayList`, `Random`
`java.io`	Input/output operations	`File`, `BufferedReader`
`java.net`	Networking	`URL`, `Socket`
`java.time`	Date and time	`LocalDate`, `LocalTime`
`java.sql`	Database connectivity using JDBC	`Connection`, `ResultSet`

Example of Using Java API Classes

```
import java.util.Scanner;

class Student {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Age = " + age);
```

```
        sc.close();
    }
}
```

Sample output:

```
Enter your age: 20
Age = 20
```

Explanation:

- `java.util` is a standard Java package.
- `Scanner` is a predefined Java API class.
- `import java.util.Scanner;` allows the program to refer to the class by its simple name.
- `nextInt()` reads an integer from the input.

Advantages of Java API

- Saves development time.
- Reduces code duplication.
- Provides tested, reusable functionality.
- Makes complex programming tasks easier.
- Promotes consistency across Java applications.

Packages in Java

A package is a namespace used to organize related Java classes, interfaces, and other types.

A package in Java is a collection of related classes and interfaces grouped under a common name to improve organization, avoid naming conflicts, and provide access control.

Types of Packages

Java packages are commonly classified into two types.

A. Built-in Packages

These are supplied by the Java platform.

Examples:

- `java.lang`
- `java.util`
- `java.io`
- `java.net`

B. User-Defined Packages

These are created by programmers to organize their own classes and interfaces.

Example:

```
package mypack;

public class Demo {
    public void display() {
        System.out.println("My package");
    }
}
```

Here, `mypack` is a user-defined package.

Advantages of Packages

1. Organize related classes.
2. Prevent naming conflicts.
3. Support access control.
4. Encourage code reuse.
5. Make large programs easier to maintain.
6. Separate program components into logical groups.

Using System Packages

What Are System Packages?

System packages are standard packages provided by the Java platform. They contain predefined classes and interfaces for common programming operations.

They are also commonly called built-in packages or Java library packages.

Common System Packages

1. `java.lang`

Provides fundamental classes used in Java programs.

Examples: `String`, `Math`, `System`, `Object`, and `Integer`.

This package is automatically imported into ordinary Java source files.

Example:

```
class Demo {
    public static void main(String[] args) {
        System.out.println(Math.sqrt(25));
        System.out.println("Hello Java".length());
    }
}
```

Output:

```
5.0
10
```

2. `java.util`

Provides utility classes and collection classes.

Examples: `Scanner`, `ArrayList`, `HashMap`, and `Random`.

```
import java.util.ArrayList;

class Demo {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();

        names.add("Amit");
        names.add("Neha");

        System.out.println(names);
    }
}
```

Output:

```
[Amit, Neha]
```

3. `java.io`

Provides classes for input/output operations.

Examples: `File`, `FileReader`, and `BufferedReader`.

4. `java.net`

Provides classes for networking and communication.

Examples: `URL` and `Socket`.

5. `java.time`

Provides classes for working with dates and times.

Example: `LocalDate`.

```
import java.time.LocalDate;

class Demo {
    public static void main(String[] args) {
        System.out.println(LocalDate.of(2026, 10, 10));
    }
}
```

Output:

```
2026-10-10
```

Ways to Use System Packages

There are three common approaches.

Method 1: Import a single class

```
import java.util.Scanner;
```

Method 2: Import accessible types from a package

```
import java.util.*;
```

Method 3: Use a fully qualified class name

```
java.util.Scanner sc = new java.util.Scanner(System.in);
```

Important: `import java.util.*;` imports accessible types directly in `java.util`; it does not import its subpackages.

Naming Conventions in Java Packages

What Are Naming Conventions?

Naming conventions are standard rules followed when naming packages, classes, interfaces, methods, and variables. They improve readability and make programs easier to understand.

Package Naming Conventions

1. Package names are generally written in lowercase letters.
2. Names should be meaningful and descriptive.
3. Avoid spaces and special characters.
4. Use reverse-domain naming for organizational packages to reduce naming conflicts.
5. Separate package-name components using dots (.).

Examples:

```
package student;
package college.records;
package com.example.bca;
```

For example, `com.example.bca` represents a hierarchy of package names.

Reverse-Domain Naming

Organizations often use their domain name in reverse order as the beginning of a package name.

For example, an organization using `example.com` might create:

```
package com.example.student;
```

This convention reduces conflicts between packages created by different organizations.

Naming Conventions for Other Java Elements

Element	Convention	Example
Package	Lowercase	`college.student`
Class	PascalCase	`StudentRecord`
Interface	PascalCase	`Printable`
Method	camelCase	`displayDetails()`
Variable	camelCase	`studentName`
Constant	Uppercase with underscores	`MAX_MARKS`

Creating Packages in Java

Creating a package means declaring a package name in a Java source file and compiling the source so the class is placed in the appropriate package structure.

The `package` keyword is used to declare a package.

Syntax

```
package packageName;

class ClassName {
    // Class members
}
```

The package declaration must appear before import declarations and type declarations.

Example: Creating a Package

Create a file named `Student.java`:

```
package college;

public class Student {
    public void display() {
        System.out.println("Student package created");
    }
}
```

Compile the file from the directory containing `Student.java`:

```
javac -d . Student.java
```

The `-d .` option tells the compiler to place generated class files in the current directory structure according to their package names.

The resulting structure is:

```
CurrentDirectory/
    Student.java
    college/
        Student.class
```

The package `college` now contains the compiled `Student` class.

Creating a Package with Multiple Classes

Two source files can belong to the same package.

File 1: `Student.java`

```
package college;

public class Student {
    public void show() {
        System.out.println("Student class");
    }
}
```

File 2: `Teacher.java`

```
package college;

public class Teacher {
    public void display() {
        System.out.println("Teacher class");
    }
}
```

Compile both files:

```
javac -d . Student.java Teacher.java
```

Both classes belong to the `college` package.

Accessing Packages in Java

Accessing a package means using classes and interfaces defined inside that package from another class or package.

Java provides different ways to access package members.

Method 1: Using the Import Statement

Import the class and use its simple name.

Example: Suppose `Student.java` belongs to package `college`.

```
package college;

public class Student {
    public void display() {
        System.out.println("Welcome to college package");
    }
}
```

Now create `Test.java` outside the `college` package:

```
import college.Student;

class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Output:

```
Welcome to college package
```

Method 2: Using a Fully Qualified Name

A fully qualified name contains the package name followed by the class name.

Example:

```
class Test {
    public static void main(String[] args) {
        college.Student s = new college.Student();
```

```
        s.display();
    }
}
```

This works when the `college.Student` class has already been compiled and is available on the classpath.

Method 3: Using the Wildcard Import

```
import college.*;
```

This allows accessible types directly in the `college` package to be referred to by their simple names.

Note: A wildcard import does not import subpackages, and importing a package does not automatically make its non-public classes accessible from outside that package.

Access Modifiers and Packages

Access modifiers determine where classes and members can be accessed.

Modifier	Same class	Same package	Subclass in another package	Unrelated class in another package
`public`	Yes	Yes	Yes, subject to inheritance rules	Yes
`protected`	Yes	Yes	Yes, subject to protected-access rules	No
Default (package-private)	Yes	Yes	No	No
`private`	Yes	No, except permitted access through enclosing-class rules	No direct access	No direct access

Exam point: A public class can be accessed from another package if the package is available and the class is imported or referenced by its fully qualified name.

Using a Package in Java

Using a package means accessing and utilizing the classes or interfaces contained in that package within a Java program.

The `import` statement is the most common way to do this.

Example Using a User-Defined Package

Suppose we create a package named `mypack`.

File 1: `Message.java`

```
package mypack;

public class Message {
```

```
    public void show() {
        System.out.println("Hello from mypack");
    }
}
```

File 2: `Test.java`

```
import mypack.Message;

class Test {
    public static void main(String[] args) {
        Message m = new Message();
        m.show();
    }
}
```

Compile and run:

```
javac -d . Message.java
javac -cp . Test.java
java -cp . Test
```

Output:

```
Hello from mypack
```

Explanation

1. The `package` statement declares `mypack`.
2. The `Message` class is declared `public`.
3. The `import` statement allows `Test` to refer to `Message` by its simple name.
4. An object of `Message` is created.
5. The `show()` method is called.

Difference Between Creating and Using a Package

Creating a Package	Using a Package
Declares a package for source types.	Accesses types already defined in a package.
Uses the `package` keyword.	Commonly uses the `import` keyword or a fully qualified name.
Example: `package mypack;`	Example: `import mypack.Message;`
Organizes classes under a package name.	Allows a program to use accessible classes from that package.

Adding a Class to a Package

Adding a class to a package means declaring the same package name in the new class's source file and compiling it into the corresponding package structure.

Important: Every source file containing a top-level class in a named package must declare that package if the class is intended to belong to it.

Example

Suppose the `college` package already contains `Student`.

Now add another class called `Result`.

File 1: `Student.java`

```
package college;

public class Student {
    public void display() {
        System.out.println("Student details");
    }
}
```

File 2: `Result.java`

```
package college;

public class Result {
    public void showResult() {
        System.out.println("Result declared");
    }
}
```

Both files belong to the same package because they declare:

```
package college;
```

Compile both:

```
javac -d . Student.java Result.java
```

The package structure becomes:

```
CurrentDirectory/
    college/
        Student.class
        Result.class
```

Accessing Both Classes

Create a file named `Test.java` outside the package:

```
import college.Student;
import college.Result;

class Test {
    public static void main(String[] args) {
        Student s = new Student();
        Result r = new Result();

        s.display();
        r.showResult();
    }
}
```

Output:

```
Student details
Result declared
```

Advantages of Adding Classes to an Existing Package

- Keeps related classes together.
- Improves modularity.
- Makes maintenance easier.
- Supports code reuse.
- Helps establish a clear project structure.

Quick Revision Sheet

Topic	Key point
Java API	Standard library of classes and interfaces.
System package	Package supplied by the Java platform.
`java.lang`	Automatically imported into ordinary Java source files.
`java.util`	Contains utilities and collection classes.
Naming convention	Package names are generally lowercase.
`package`	Declares the package to which a source type belongs.
`import`	Allows accessible types to be referenced by simple names.
Fully qualified name	Package name plus class name.
Creating a package	Declare the package and compile into the proper structure.
Accessing a package	Import or fully qualify an accessible type.
Adding a class	Declare the same package in the new source file.
`javac -d .`	Places compiled classes in the corresponding package directory structure.

Class Hiding and Data Hiding in Java

Data hiding is an OOP concept in which the internal data of a class is protected from direct access by outside code. In Java, it is commonly achieved using the `private` access modifier.

Advantages of data hiding

1. Protects data from unauthorized access.
2. Improves security.
3. Supports encapsulation.
4. Allows controlled access through getter and setter methods.
5. Makes programs easier to maintain.

Example

```
class Student {
    private int marks;

    public void setMarks(int m) {
        if (m >= 0 && m <= 100) {
            marks = m;
        }
    }

    public int getMarks() {
        return marks;
    }
}

class Main {
    public static void main(String[] args) {
        Student s = new Student();
```

```
        s.setMarks(85);
        System.out.println(s.getMarks());
    }
}
```

Output:

```
85
```

Explanation: The variable `marks` is declared `private`, so it cannot be accessed directly from outside the `Student` class. It is accessed through public methods.

Class Hiding

In Java, class hiding is not a general OOP mechanism in the same sense as data hiding. A related concept occurs when a nested class in a subclass has the same name as a nested class in its superclass. The subclass's nested class hides the superclass's nested class by name.

For most introductory OOP exam answers, if the question refers to protecting class data, explain data hiding and encapsulation.

Arrays in Java

An array is an object that stores a fixed number of values of the same data type. Each value is accessed using an index, beginning at `0`.

For example, an integer array can store several integer values under one variable name.

Features of arrays

- Store multiple values of the same declared type.
- Use zero-based indexing.
- Have a fixed length after creation.
- Can store primitive values or object references.
- Are objects in Java and have a `length` field.

Types of arrays

1. One-dimensional array
2. Two-dimensional array
3. Multidimensional array

One-dimensional array

Syntax

```
dataType[] arrayName = new dataType[size];
```

Example

```
class Main {
    public static void main(String[] args) {
        int[] marks = {75, 80, 90, 85};

        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
```

```
        }
    }
}
```

Output:

```
75
80
90
85
```

Here, `marks.length` gives the number of elements in the array.

Two-dimensional array

A two-dimensional array is an array whose elements are themselves arrays. It is commonly used to represent tables and matrices.

Example

```
class Main {
    public static void main(String[] args) {
        int[][] a = {
            {1, 2, 3},
            {4, 5, 6}
        };

        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[i].length; j++) {
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
1 2 3
4 5 6
```

Advantages and limitations

Advantages

- Convenient storage of multiple values.
- Fast access through an index.
- Useful for sorting, searching, and matrix operations.

Limitations

- The length cannot be changed after the array is created.
- An array stores elements of one declared component type.
- Inserting or removing elements in the middle often requires shifting elements.

Strings in Java

A String is a sequence of characters. Java provides the `String` class in the `java.lang` package to represent and manipulate strings.

Examples:

```
String name = "Rahul";
String city = "Bareilly";
```

Strings are objects in Java.

Creating strings

There are two common ways to create a string.

1. Using a string literal

```
String s1 = "Hello";
```

2. Using the `new` keyword

```
String s2 = new String("Hello");
```

String literals may reuse objects from the string pool, whereas `new String(...)` explicitly creates a new `String` object.

Important properties of strings

- Strings can contain letters, digits, spaces, and symbols.
- The `String` class is immutable.
- Strings support methods for searching, comparing, extracting, and replacing characters.
- The `String` class belongs to the `java.lang` package, which is automatically available.

Example

```
class Main {
    public static void main(String[] args) {
        String name = "Java Programming";

        System.out.println(name);
        System.out.println(name.length());
        System.out.println(name.charAt(0));
    }
}
```

Output:

```
Java Programming
16
J
```

Comparing strings

The `equals()` method compares string contents, whereas == compares object references.

```
class Main {
    public static void main(String[] args) {
        String s1 = new String("Java");
        String s2 = new String("Java");
```

```
        System.out.println(s1 == s2);
        System.out.println(s1.equals(s2));
    }
}
```

Output:

```
false
true
```

Explanation: Both strings contain the same text, but they are different objects. Therefore, `==` returns `false`, while `equals()` returns `true`.

String Handling in Java

String handling means performing operations on strings, such as concatenation, comparison, searching, extraction, and modification.

Java provides three important classes for working with text:

`String`

Immutable. Operations that appear to modify a string return a new string.

`StringBuffer`

Mutable and synchronized, making its methods suitable for thread-safe use when shared appropriately.

`StringBuilder`

Mutable and generally preferred for string modification in single-threaded code because it does not synchronize its methods.

Important String methods

Method	Purpose	Example
`length()`	Returns string length	`"Java".length()` → `4`
`charAt(i)`	Returns character at index `i`	`"Java".charAt(1)` → `'a'`
`equals(s)`	Compares contents	`"Hi".equals("Hi")` → `true`
`equalsIgnoreCase(s)`	Compares ignoring case	`"JAVA".equalsIgnoreCase("java")` → `true`
`compareTo(s)`	Lexicographically compares strings	`"A".compareTo("B")` → negative value
`concat(s)`	Joins strings	`"Hello".concat(" Java")`
`substring(a)`	Extracts from index `a`	`"Programming".substring(3)` → `"gramming"`
`substring(a,b)`	Extracts indices `a` through `b-1`	`"Programming".substring(0,4)` → `"Prog"`
`indexOf(s)`	Finds first occurrence	`"banana".indexOf("na")` → `2`
`replace(a,b)`	Replaces matching characters or sequences	`"Java".replace('a','o')` → `"Jovo"`
`toUpperCase()`	Converts to uppercase	`"java".toUpperCase()` → `"JAVA"`
`toLowerCase()`	Converts to lowercase	`"JAVA".toLowerCase()` → `"java"`

`trim()`	Removes leading/trailing characters recognized as whitespace by `trim()`	`" hi ".trim()` → `"hi"`

String concatenation

Concatenation means joining two or more strings.

```
class Main {
    public static void main(String[] args) {
        String first = "Hello";
        String second = "Java";

        String result = first + " " + second;
        System.out.println(result);
    }
}
```

Output:

```
Hello Java
```

StringBuffer example

```
class Main {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");

        sb.append(" Java");
        sb.insert(5, " World");
        System.out.println(sb);
    }
}
```

Output:

```
Hello World Java
```

The `append()` method adds text at the end, while `insert()` adds text at the specified position.

StringBuilder example

```
class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");

        sb.append(" Language");
        sb.reverse();

        System.out.println(sb);
    }
}
```

Output:

```
egaugnaL avaJ
```

Exam point: `String` is immutable, while `StringBuffer` and `StringBuilder` are mutable.

Vector in Java

A Vector is a growable array-like collection that can increase or decrease in size as elements are added or removed. It belongs to the `java.util` package.

Unlike an ordinary array, a Vector can grow dynamically.

Features of Vector

- Stores multiple objects.
- Grows dynamically when elements are added.
- Maintains insertion order.
- Allows duplicate elements.
- Supports indexed access.
- Its individual methods are synchronized.

Creating a Vector

```
import java.util.Vector;

class Main {
    public static void main(String[] args) {
        Vector<String> names = new Vector<String>();

        names.add("Amit");
        names.add("Ravi");
        names.add("Neha");

        System.out.println(names);
    }
}
```

Output:

```
[Amit, Ravi, Neha]
```

Here, `Vector<String>` specifies that the Vector stores strings.

Important Vector methods

Method	Purpose
`add(element)`	Adds an element
`addElement(element)`	Adds an element using a legacy Vector method
`get(index)`	Returns the element at an index
`elementAt(index)`	Returns the element at an index
`set(index, element)`	Replaces an element
`remove(index)`	Removes the element at an index
`removeElement(element)`	Removes the first matching element
`size()`	Returns the number of elements
`capacity()`	Returns the current capacity
`firstElement()`	Returns the first element
`lastElement()`	Returns the last element
`clear()`	Removes all elements

Example using Vector methods

```
import java.util.Vector;

class Main {
    public static void main(String[] args) {
        Vector<Integer> v = new Vector<Integer>();

        v.add(10);
        v.add(20);
        v.add(30);

        System.out.println("Vector: " + v);
        System.out.println("First: " + v.firstElement());
        System.out.println("Size: " + v.size());

        v.remove(1);
        System.out.println("After removal: " + v);
    }
}
```

Output:

```
Vector: [10, 20, 30]
First: 10
Size: 3
After removal: [10, 30]
```

Important distinction: `size()` returns the number of stored elements, whereas `capacity()` returns the current storage capacity. A Vector's capacity may be greater than its size.

Wrapper Classes in Java

A wrapper class converts a primitive data value into an object representation. Java provides a wrapper class for every primitive type.

For example, `int` is a primitive type, while `Integer` is its wrapper class.

Primitive types and their wrapper classes

Primitive type	Wrapper class
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

All these wrapper classes are in the `java.lang` package.

Why are wrapper classes needed?

1. To represent primitive values as objects.
2. To use primitive values with collections such as `ArrayList<Integer>` and `Vector<Integer>`.
3. To access utility methods such as `Integer.parseInt()`.
4. To support conversion between primitive values and their object forms.

5. To represent a missing value using `null` when an object reference is appropriate. Primitive variables themselves cannot hold `null`.

Boxing and unboxing

Boxing means converting a primitive value into its corresponding wrapper object. Unboxing means converting a wrapper object back into a primitive value.

Java supports autoboxing and auto-unboxing, which perform these conversions automatically in many contexts.

Example

```
class Main {
    public static void main(String[] args) {
        int a = 10;

        Integer obj = a;  // Autoboxing
        int b = obj;      // Auto-unboxing

        System.out.println(obj);
        System.out.println(b);
    }
}
```

Output:

```
10
10
```

Equivalent explicit conversions can be written as:

```
Integer obj = Integer.valueOf(10);
int b = obj.intValue();
```

Useful wrapper class methods

Method	Purpose	Example
`Integer.parseInt(s)`	Converts a numeric string to primitive `int`	`Integer.parseInt("123")` → `123`
`Integer.valueOf(s)`	Converts a numeric string to an `Integer` object	`Integer.valueOf("123")`
`Double.parseDouble(s)`	Converts a numeric string to primitive `double`	`Double.parseDouble("3.5")` → `3.5`
`String.valueOf(x)`	Converts a value to a string representation	`String.valueOf(25)` → `"25"`
`Integer.toString(n)`	Converts an integer to a string	`Integer.toString(50)` → `"50"`

Example of parsing a string

```
class Main {
    public static void main(String[] args) {
        String s = "100";
        int n = Integer.parseInt(s);

        System.out.println(n + 20);
    }
```

```
}
```

Output:

```
120
```

The string `"100"` is converted into the integer `100`, so arithmetic addition is performed.

Advantages and limitations

Advantages

- Allow primitive values to be used in object-based APIs and collections.
- Provide conversion and utility methods.
- Can represent absence with `null` in object-based data structures.

Limitations

- Wrapper objects generally require more memory than primitive values.
- Conversions can introduce overhead.
- Unboxing a `null` wrapper reference throws a `NullPointerException`.

Example of the last point:

```
Integer x = null;
// int y = x; // Throws NullPointerException
```

Quick Revision Table

Topic	Key point
Data hiding	Protecting class data using access control, commonly `private`
Array	Fixed-length, indexed collection
String	Immutable sequence of characters
StringBuffer	Mutable, synchronized string-handling class
StringBuilder	Mutable, non-synchronized string-handling class
Vector	Dynamically growing, synchronized collection
Wrapper class	Object representation of a primitive type
Autoboxing	Primitive to wrapper object
Auto-unboxing	Wrapper object to primitive